

1) Compiler Phases for

Expression :

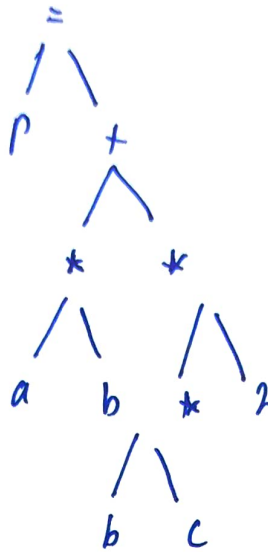
$$P = (a * b) + (b * c) * 2$$

(a) Tokens (Lexical Analysis)

Token	Type
P	Identifier
=	Assignment operator
(	Left Parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right Parenthesis
+	Addition operator
(	Left Parenthesis
b	Identifier
*	Multiplication operator
c	Identifier
)	Right Parenthesis
*	Multiplication operator
2	Integer constant

$$\text{Total Tokens} = 15$$

(b) Syntax Tree :



(c) Intermediate code (Three Address Code) :

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

(d) optimized code :

$$t_1 = a * b$$

$$t_2 = b * c$$

$$P = t_1 + (t_2 * 2)$$

(e) Target code : (~~Example~~)

load R_1, a

mul R_1, b

store $t,$

load R_2, b

mul R_2, c

mul $R_2, 2$

Add R_1, R_2

store p

2) Five Valid strings for Each Regular Expression :

(a) $(a^*)^*$

Language : Any number of a's

Examples :

* ϵ

* a

* a a

* a a a

* a a a a

(b) $(b^*)^* a b b$

Language : Any number of b's followed by abb

Example :

* abb

* babb

* bbabb

* bbbabb

* bbbbabb

(c) $(a^*b^*)^*$

Language : Any combination of a and b

Example :

$\Rightarrow \epsilon$

$\Rightarrow ab$

$\Rightarrow aaabbb$

$\Rightarrow aabb$

$\Rightarrow bbaa$

(d) $(ab)^*$

Language : Repetition of ab

Example :

$\Rightarrow \epsilon$

$\Rightarrow ab$

$\Rightarrow abab$

$\Rightarrow ababab$

$\Rightarrow abababab$

(e) $(aba)^* + (a^*b^*)^*$

Examples :

* aba

* abaaba

* aaabbb

* aabb

* ϵ

3) Lexical Analysis :

(i) Number of Tokens

Program :

```
main ()
```

```
{
```

```
int a = 10 ;
```

```
char b = "abc" ;
```

```
int c = 30 ;
```

```
char d = "xyz" ;
```

```
in /* comment */ t m = 40.5 ;
```

```
}
```

Valid Tokens :

```
main () { int a = 10 ;
```

```
char b = "abc" ;
```

```
c = 30 ;
```

d = "xyz" ;

m = 40.5 ;

3

Total Tokens = 24

(ii) Lexical Errors :

Program :

```
void main ()
```

```
{
```

```
int x = 10, y = 20 ;
```

```
char * a ;
```

```
a = &x ;
```

```
x = 1xab ;
```

```
}
```

Errors :

1. 1xab → invalid hexadecimal constant.
2. a = &x ; → lexically valid (semantic / type issue, not a lexical error).

4) Regular Expression :

(i) Even number of 0's :

$1^* (01^* 01^*)^*$

(ii) At least one 1

$$(0+1) * 1 (0+1) *$$

(iii) odd number of both 0's and 1's

$$1 * (01 * 01 *) * 01 * (10 * 10 *) * 10 *$$

(iv) starts with 1 and ends with 0

$$1(0+1) * 0$$

(v) No consecutive 1's

$$(0+10) * 1 ?$$

5) Lexical Analyser :

Input :

int var123 = 12345 ;

float var456 = 123.45 ;

if (var123 == var456)

{
 return ;

}

(a) Number of tokens

Token	Type
int	keyword
Var 123	identifier
=	operator
12345	integer
;	separator
float	keyword
Var 456	identifier
=	operator
123.45	float
;	separator
if	keyword
(	separator
Var 123	identifier
==	operator
Var 456	identifier
)	separator
{	separator
return	keyword
;	separator
}	separator

Total Tokens = 20

(b) Token Categories :

Keywords :

- * int
- * float
- * if
- * return

operator :

- =
- =
- =

Integer Literal :

12345

Identifiers :

- Var 123
- Var 456
- Var 123
- Var 456

separator :

- ;
- ;
- {
- }
- {
- ;
- }

Floating Literal

123.45

(c) FSM state Transitions (Approximate) :

* Keywords $\rightarrow$ 18 transitions

* Identifier $\rightarrow$ 24 transitions

* Integer literal $\rightarrow$ 5 transitions

* Float literal $\rightarrow$ 7 transitions

* operators $\rightarrow$ 4 transitions

* separators $\rightarrow$ 7 transitions

Approximate Total FSM State Transitions = 65